Объектно-ориентированное программирование

4 семестр
Лекция 3.

Содержание лекции:

- Тестирование
- Виды ошибок
- Уровни тестирования
- Модульные тесты NUnit

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Тестирование

Тести́рование програ́ммного обеспе́че́ния — процесс исследования, испытания программного продукта, имеющий своей целью проверку соответствия между реальным поведением программы и её ожидаемым поведением на конечном наборе тестов, выбранных определённым образом.

Верификация (verification) – Соответствие продукта требованиям (**спецификации**). Т.е. Верификация это проверка того, что программа работает так, как было задумано.

Валидация (validation) – Соответствие продукта потребностям пользователей. Т.е. работа программы устраивает заказчика.

Виды ошибок:

Error – это ошибка **пользователя**, то есть он пытается использовать программу иным способом (например, вводит буквы в поля, где требуется вводить цифры). В качественной программе предусмотрены такие ситуации и выдаются сообщение об ошибке.

Bug (defect) – это ошибка **программиста** (или дизайнера или ещё кого, кто принимает участие в разработке), то есть когда в программе, что-то идёт не так, как планировалось. Например, внутри программа построена так, что изначально не соответствует тому, что от неё ожидается.

Failure – это сбой в работе компонента, всей программы или системы (может быть как аппаратным, так и вызванным дефектом).

Уровни тестирования:

- **Тестирование компонентов** — тестируется минимально возможный для тестирования компонент, например, отдельный класс или функция. Часто тестирование компонентов осуществляется разработчиками программного обеспечения.
- **Интеграционное тестирование** — тестируются интерфейсы между компонентами, подсистемами или системами. При наличии резерва времени на данной стадии тестирование ведётся итерационно, с постепенным подключением последующих подсистем.
- **Нагрузочное тестирование** — подвид тестирования производительности, сбор показателей и определение производительности и времени отклика программы в ответ на внешний запрос с целью установления соответствия требованиям, предъявляемым к данному ПО.

- **Системное тестирование** — тестируется интегрированная система на её соответствие требованиям.
  - **Альфа-тестирование** — является ранней версией программного продукта, тестирование которой проводится внутри организации-разработчика; может быть вероятно частичное привлечение конечных пользователей.
  - **Бета-тестирование** — практически готовое ПО, выпускаемое для ограниченного количества пользователей, разрабатывается в первую очередь для тестирования конечными пользователями и получения отзывов клиентов о продукте для внесения соответствующих изменений.

НЕКОТОРЫЕ ТЕХНИКИ ТЕСТ-ДИЗАЙНА

1. **Эквивалентное Разделение** (Equivalence Partitioning) — это техника, при которой функционал (часто диапазон возможных вводимых значений) разделяется на группы эквивалентных по своему влиянию на систему значений. ПРИМЕР: есть диапазон допустимых значений от 1 до 10, выбирается одно верное значение внутри интервала (например, 5) и одно неверное значение вне интервала — 0.
2. **Анализ Граничных Значений** (Boundary Value Analysis) — это техника проверки поведения продукта на крайних (граничных) значениях входных данных. Если брать выше ПРИМЕР: берется минимальная и максимальная границы (1 и 10), и значения больше и меньше границ (0 и 11).

3. **Доменный анализ** (Domain Analysis Testing) — это техника основана на разбиении диапазона возможных значений переменной на поддиапазоны, с последующим выбором одного или нескольких значений из каждого домена для тестирования.
4. **Причина / Следствие** (Cause/Effect — CE). Подразумевается ввод условий, для получения ответа от системы (следствие).
5. **Сценарий использования** (Use Case Testing) — описывает сценарий взаимодействия двух и более участников (как правило — пользователя и системы).

Тестирование компонентов

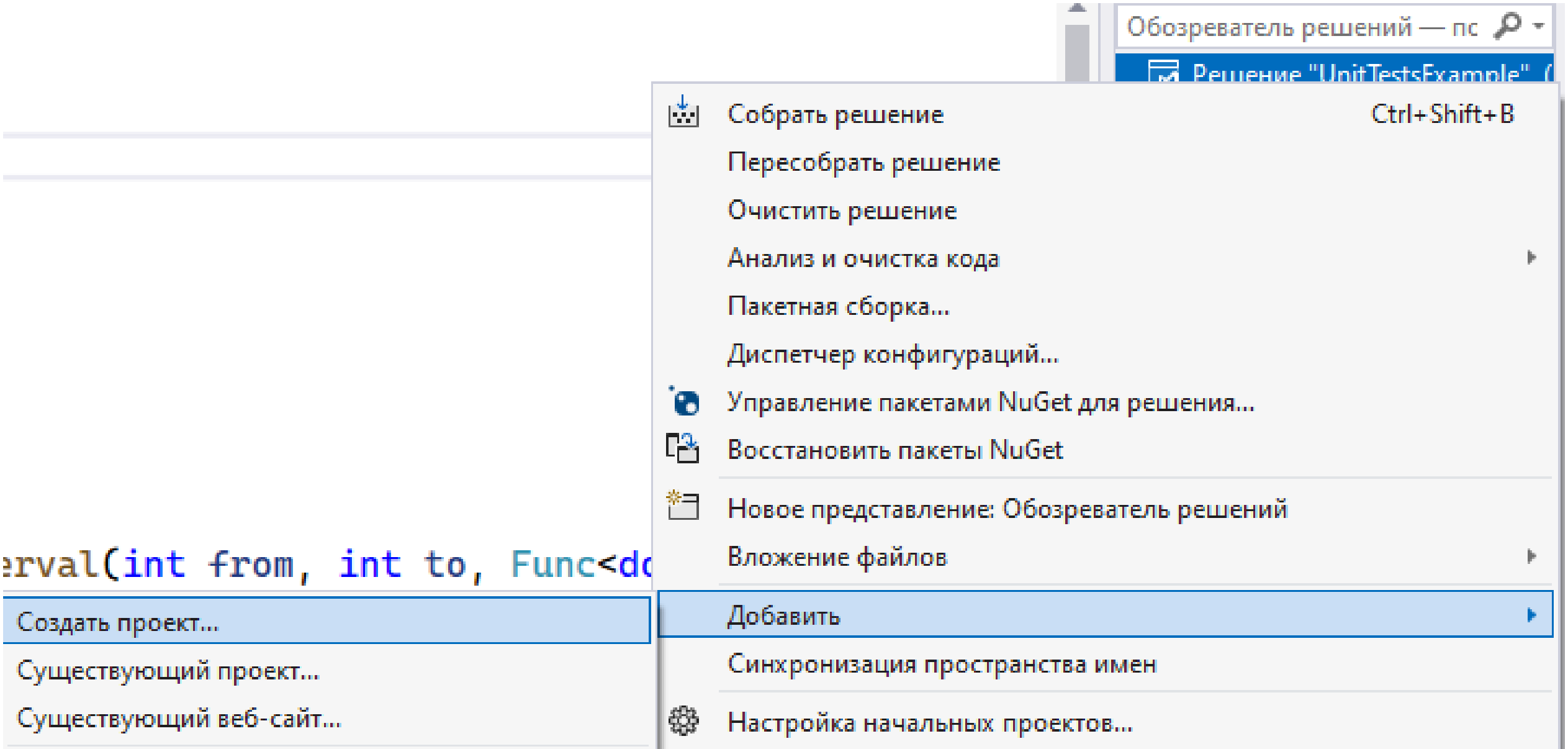
Модульное (компонентное) тестирование **(Unit Testing)** проводится самими разработчиками, т.к. предполагает полный доступ к коду, для тестирования какого-либо одного логически выделенного и изолированного элемента (модуля) системы в коде, проверяет функциональность и ищет дефекты в частях приложения, которые доступны и могут быть протестированы по-отдельности (модули программ, объекты, классы, функции и т.д.).

Создадим WindowsForms проект, добавим класс поиска целочисленных корней уравнения, который будем тестировать:

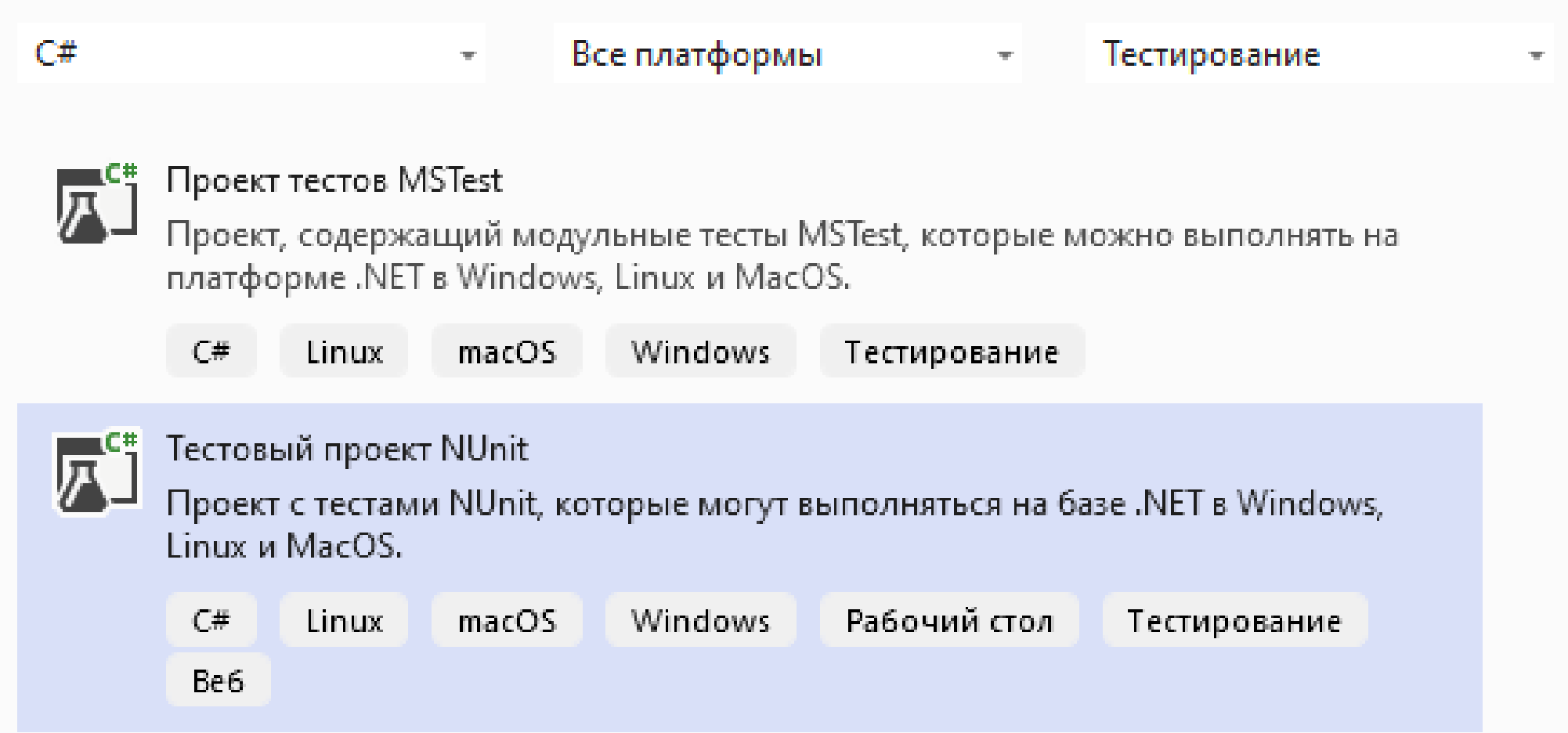
```
public class IntegerEquationSolver
    {
        public List<int> GetRootsOnInterval(int from, int to,
        Func<double, double> function)
        {
            var roots = new List<int>();

            for (int x = from; x <= to; x++)
            {
                if (Math.Abs(function(x)) <= 1e-12)
                    roots.Add(x);
            }
            return roots;
        }
    }
```

Добавим к решению тесты. Необходимо нажать ПКМ на Решение –>

Добавить –> Создать проект –>:



Выбрать C#, Все платформы, Тестирование. Из отобразившегося списка проектов выбрать «Тестовый проект NUnit»:

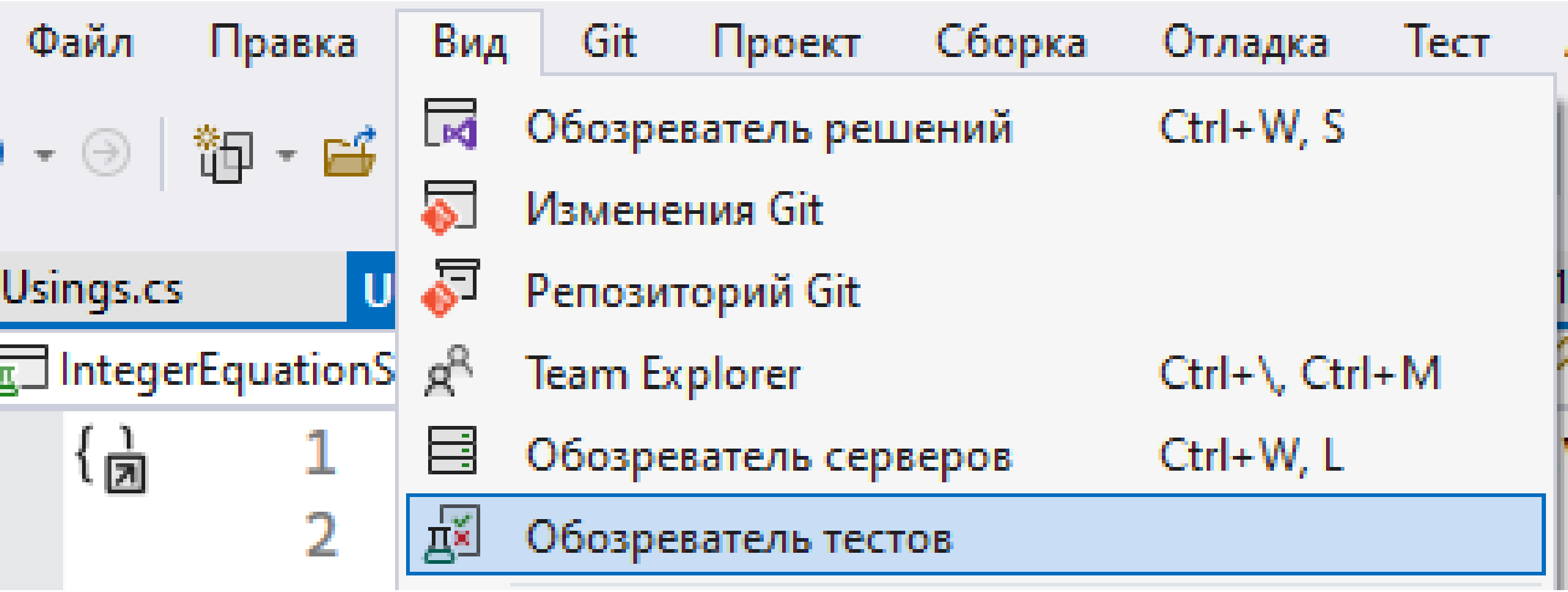


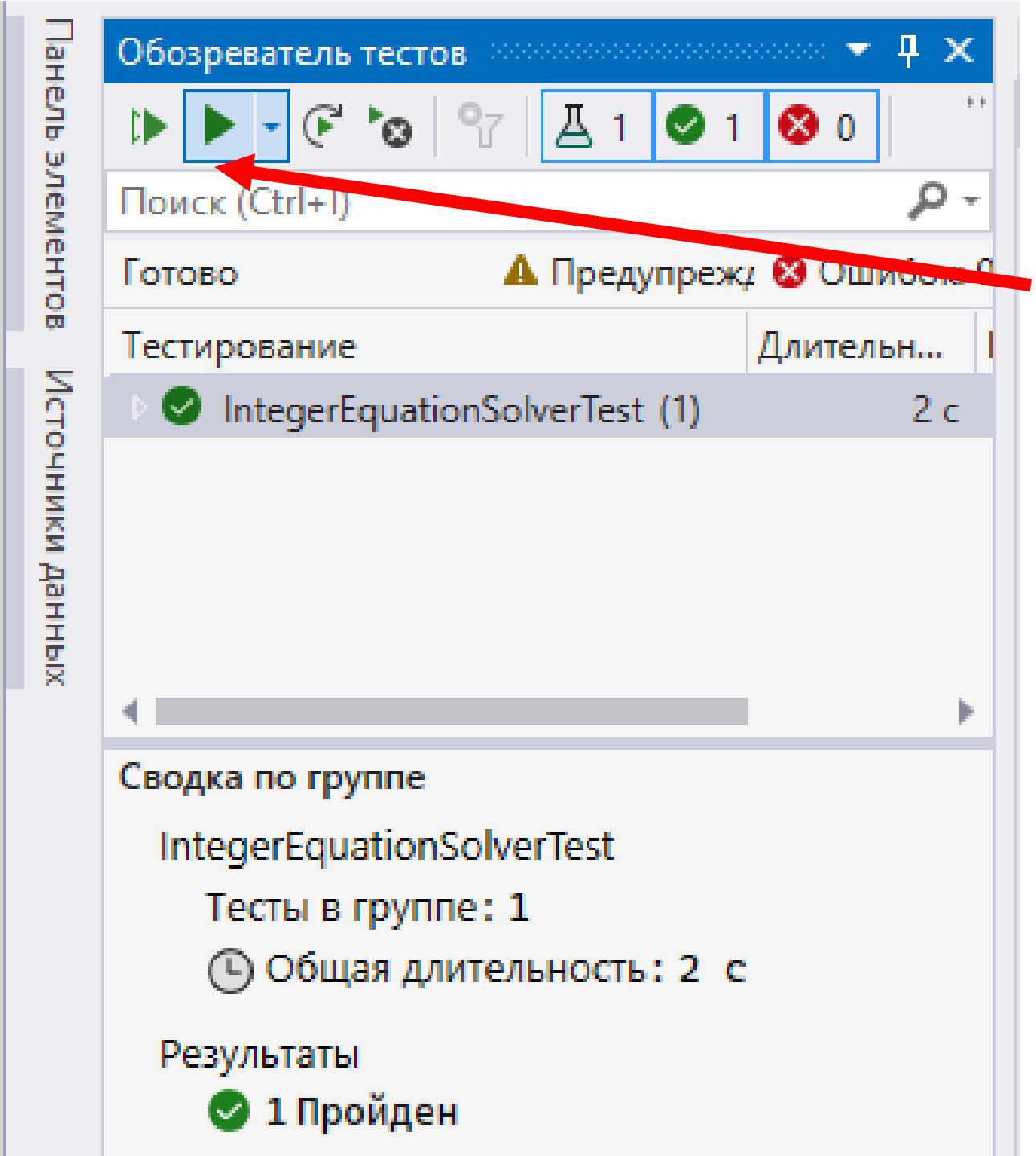
Подтверждаем и выбираем имя проекту, обычно тесты называются из названия класса, который будет тестироваться с припиской Test.

Содержимое проекта тестов:

```
namespace IntegerEquationSolverTest
{
    public class Tests
    {
        [SetUp]                 Запускается один раз перед запуском тестов
        public void Setup()
        {
        }

        [Test]                  Собственно сам тест
        public void Test1()
        {
            Assert.Pass();  Заглушка, говорит, что тест пройден
        }
    }
}
```

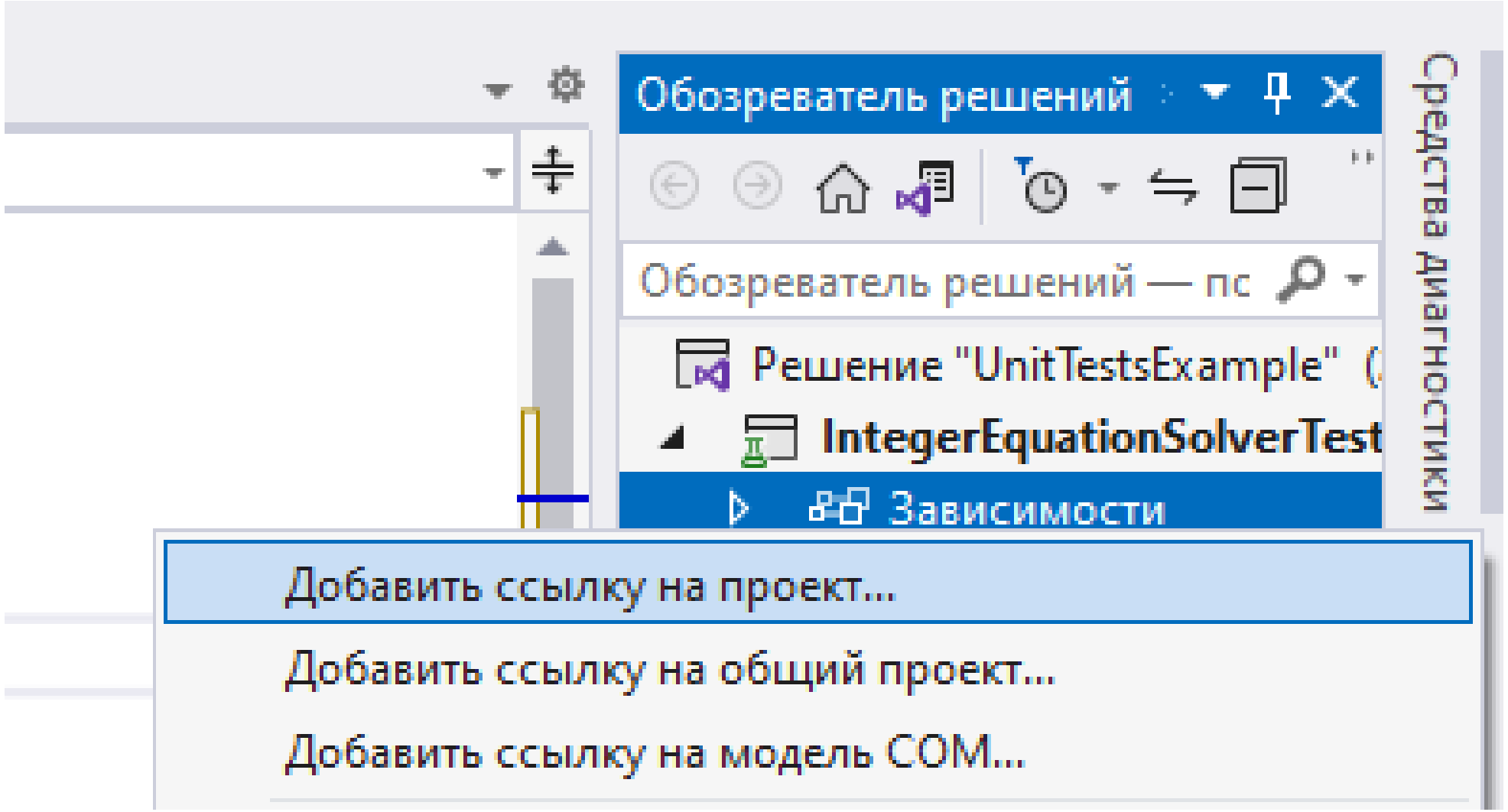
Далее выберем: Вид -> обозреватель тестов.



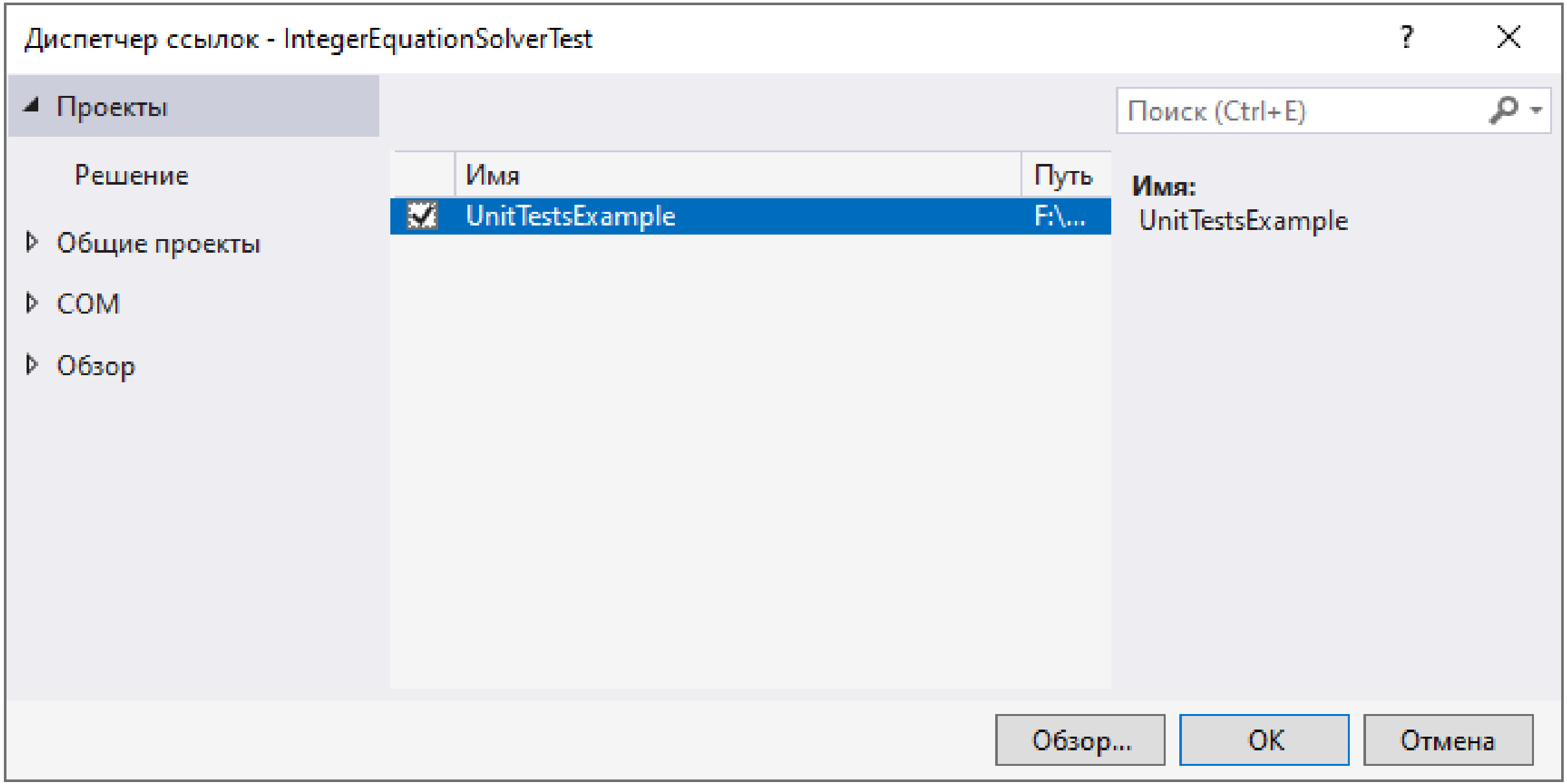


В данном окне отображается основная информация по тестам, здесь находятся кнопки запуска и отладки тестов.

Создадим тесты для нашего класса поиска корней. Во-первых необходимо подключить наш проект к проекту тестов. Нажимаем ПКМ на Зависимости -> Добавить ссылку на проект.



Выбираем нужный проект из списка.

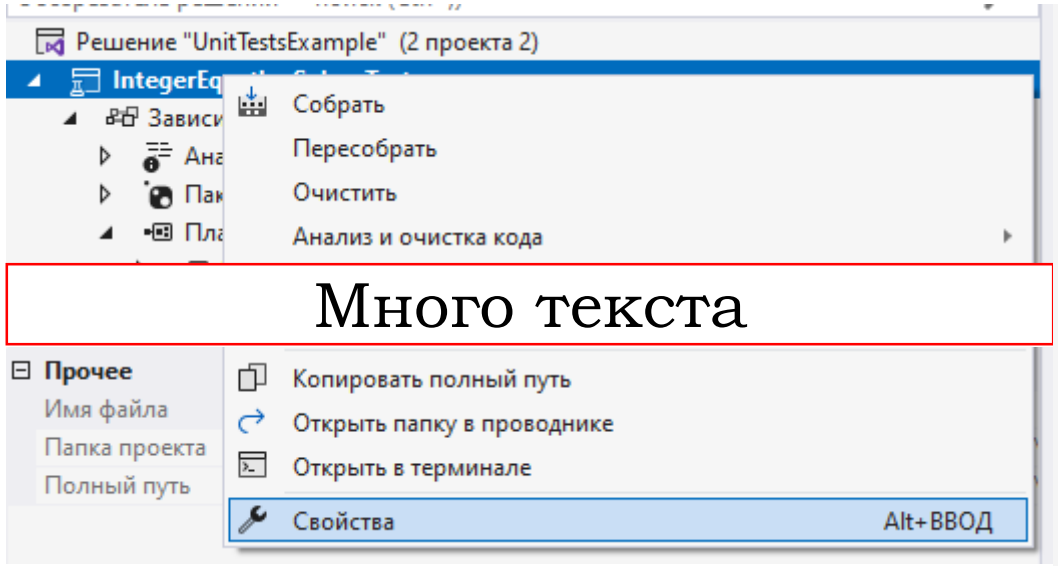


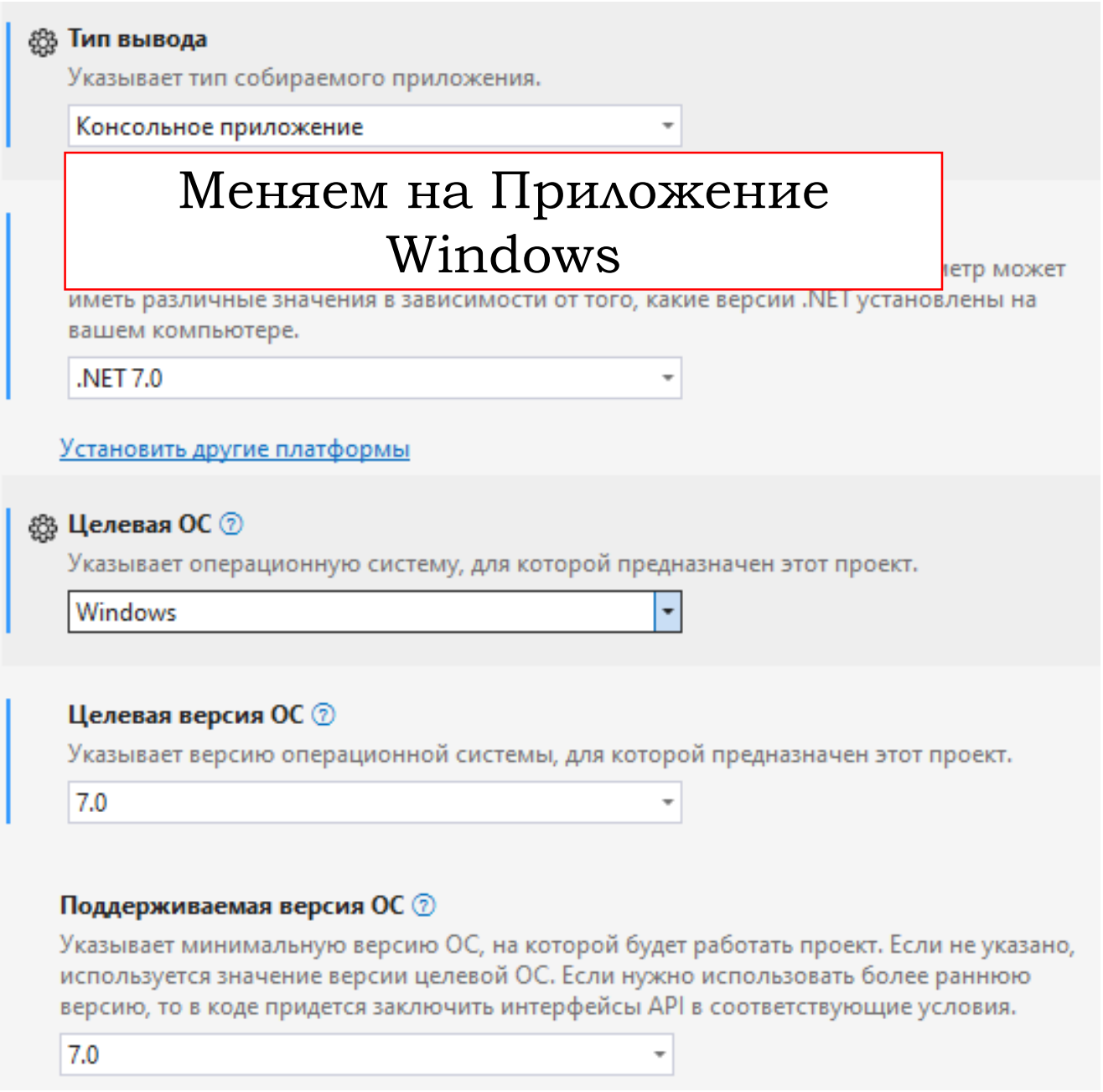
Будет высвечиваться ошибка:

⊗		Сбой при восстановлении пакета. Откат изменений пакета для «IntegerEquationSolverTest».
⊗	NU1201	Проект UnitTestsExample несовместим с net7.0 (.NETCoreApp,Version=v7.0). Проект UnitTestsExamp
⊗	NU1201	Проект UnitTestsExample несовместим с net7.0 (.NETCoreApp,Version=v7.0). Проект UnitTestsExamp
⊗		Проект "..\UnitTestsExample\UnitTestsExample.csproj" предназначен для целевой платформы "net7

Далее необходимо сменить тип приложения тестов с консольного на Windows Forms и выбрать целевую ОС. Нажимаем ПКМ на проект с тестами и выбираем Свойства (можно через комбинацию ALT + Enter).

Нажимаем ПКМ на проект с тестами и выбираем Свойства.

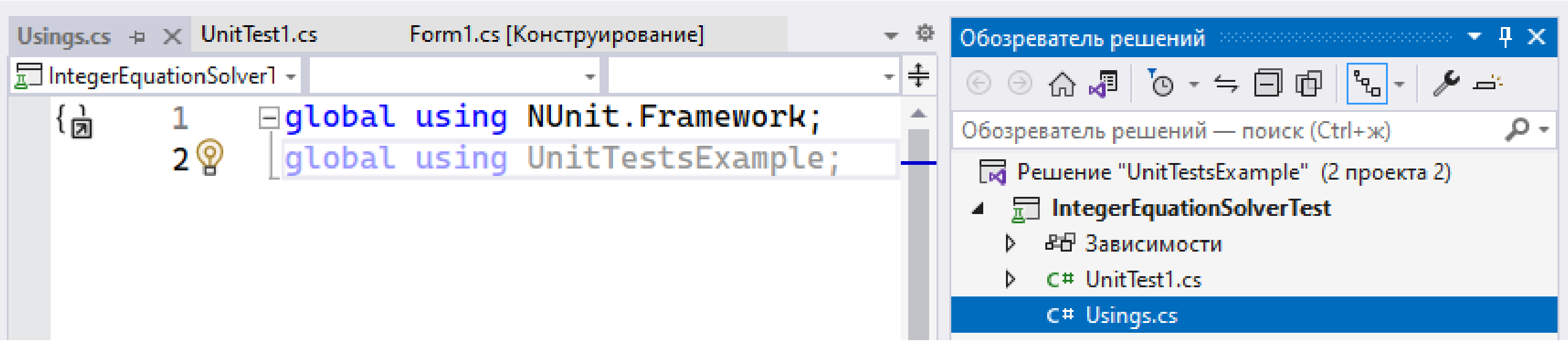




Примечание:

Консольное приложение все равно само выставится обратно, но ошибка будет решена.

Переходим в файл Usings и подключаем ссылку на наш проект.



Подготовительные моменты выполнены, можно писать тесты.

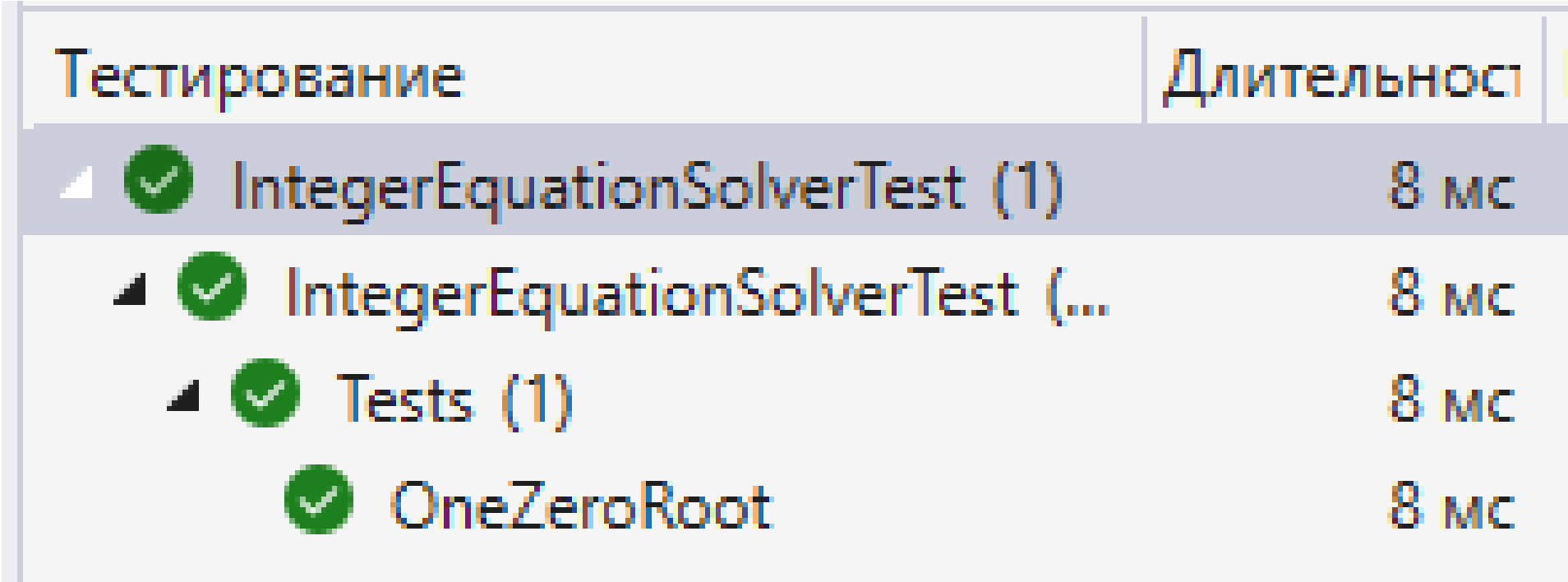
Создадим экземпляр класса для поиска корней уравнения и напишем тест «OneZeroRoot» в котором будем проверять случай, когда корень единственный и равен 0:

```
IntegerEquationSolver solver;
[SetUp]
public void Setup()
{
   solver = new IntegerEquationSolver();
}
[Test]
public void OneZeroRoot()
{
   var roots = solver.GetRootsOnInterval(-10, 10, (x) => x * x);

   Assert.AreEqual(1, roots.Count);
   Assert.AreEqual(0, roots.First());
}
```

Рассмотрим код:

```
Assert.AreEqual(expected, actual);
```

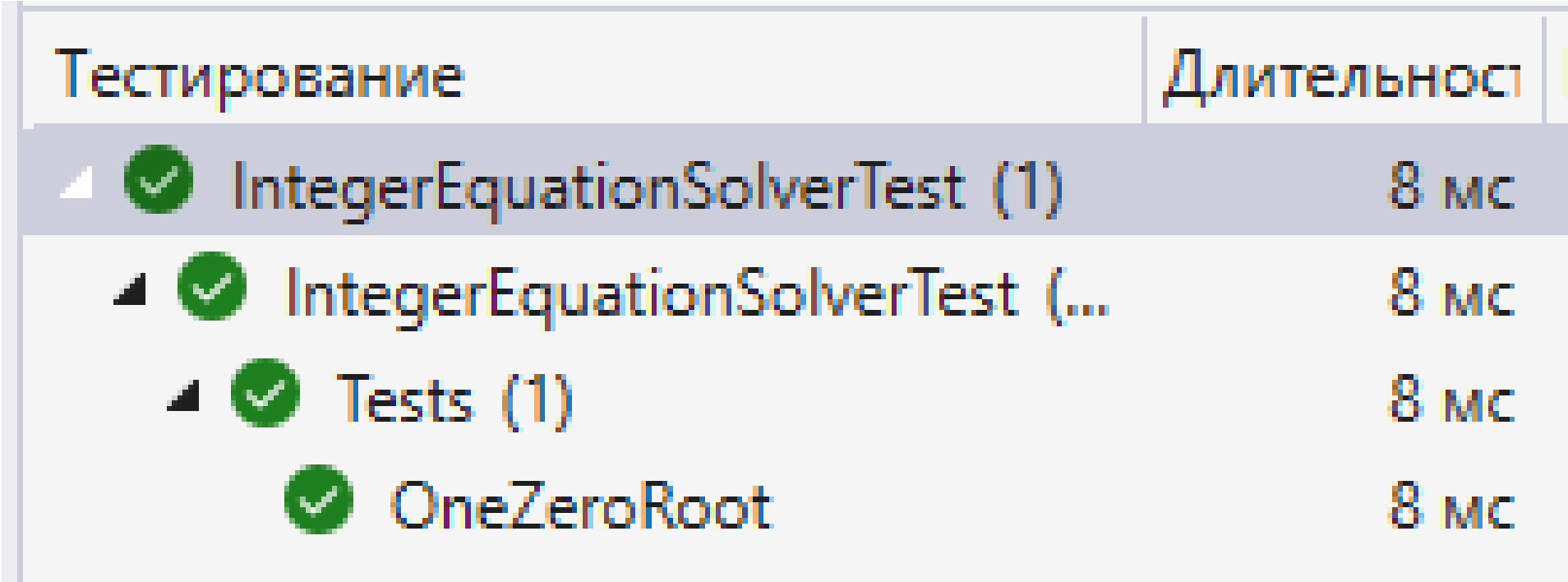
С помощью метода AreEqual мы сравниваем ожидаемый (верный) результат с полученным (actual). Тест считается пройденным, если значения совпадают.



Рассмотрим код:

```
Assert.AreEqual(expected, actual);
```

С помощью метода AreEqual мы сравниваем ожидаемый (верный) результат с полученным (actual). Тест считается пройденным, если значения совпадают.



Добавим тесты с аргументами:

```
[TestCase(-2, 2, 0)]
[TestCase(-4, 0, 1)]
[TestCase(0, 4, 1)]
[TestCase(-4, 4, 2)]
public void RootsCount(int from, int to, int expectedRootsCount)
{
   var roots = solver.GetRootsOnInterval(from,to,(x) => x * x - 9);
   Assert.AreEqual(expectedRootsCount, roots.Count);
}
```

Тестирование	Длительност
IntegerEquationSolverTest (5)	8 мс
IntegerEquationSolverTest (...	8 мс
Tests (5)	8 мс
RootsCount (4)	< 1 мс
RootsCount(0,4,1)	< 1 мс
RootsCount(-2,2,0)	< 1 мс
RootsCount(-4,0,1)	< 1 мс
RootsCount(-4,4,2)	< 1 мс
OneZeroRoot	8 мс

Добавим тесты со сравнением массивов (нужно обратить внимание, что используется **CollectionAssert** вместо **Assert**):

```
[TestCase(-4, 0, new int[] { -3 }, TestName = "Root -3")]
[TestCase(0, 4, new int[] { 3 }, TestName = "Root 3")]
[TestCase(-4, 4, new int[] { -3, 3 }, TestName = "Roots -3 3")]
public void RootsValues(int from, int to, int[] expectedRoots)
{
   var roots = solver.GetRootsOnInterval(from,to,(x) => x * x - 9);

   CollectionAssert.AreEquivalent(expectedRoots, roots.ToArray());
}
```

TestName можно указать, чтобы дать свое имя тесту.
`AreEquivalent` – проверка на то, что коллекция содержит только нужные элементы без учета порядка;
`AreEqual` – проверка на то, что коллекция содержит только нужные элементы с учетом порядка.

Так же есть следующие методы:

AllItemsAreInstancesOfType – проверка типа значений

AllItemsAreNotNull – проверка значений на Null

AllItemsAreUnique – проверка на уникальность значений в коллекции

AreNotEqual – проверка на то, что коллекция НЕ содержит заданные элементы;

AreNotEquivalent - проверка на то, что коллекция НЕ содержит заданные элементы или их порядок другой.

Contains – проверка на то, что коллекция содержит заданный элемент;

DoesNotContain – проверка на то, что коллекция НЕ содержит заданный элемент;

IsOrdered - проверка на то, что элементы коллекции упорядочены;

IsEmpty – пустая ли коллекция;

IsNotEmpty – НЕ пустая ли коллекция;

IsSubsetOf – является ли коллекция подмножеством другой коллекции;

IsNotSubsetOf – НЕ является ли коллекция подмножеством другой коллекции;

IsSupersetOf – является ли другая коллекция подмножеством данной;

IsNotSupersetOf – НЕ является ли другая коллекция подмножеством данной.

Примечание по проектированию программы

При разработке сложных приложений бывает удобно начинать разработку с написания тестов. После написания тестов пишется программа, которая будет проходить данные тесты.

При решении вариантов можно пользоваться данным подходом.

Или же пользоваться итеративной разработкой: разработать частично рабочую программу, написать тесты, доработать программу, написать ещё тесты и т.д. циклично.